

Jupyter Notebook

Jupyter Notebook là môi trường lập trình tương tác cho phép:

- Viết code Python theo từng cell, chạy từng cell độc lập
- Hiển thị kết quả ngay bên dưới cell
- Kết hợp code, text, hình ảnh + biểu đồ (visualisations) trong cùng một file có đuôi: .ipynb

VS Code hỗ trợ Jupyter thông qua Jupyter Extension của Microsoft.

Cấu trúc hoạt động

Jupyter Notebook hoạt động dựa trên hai thành phần chính:

- Notebook Document: Là file .ipynb chứa code, markdown, output và metadata, được lưu dưới định dạng JSON.
- Kernel: Là tiến trình chịu trách nhiệm thực thi mã nguồn. Kernel nhận lệnh từ Notebook, thực thi mã và trả kết quả về giao diện người dùng.

Jupyter Notebook vs file Python (.py)

Tiêu chí	Jupyter Notebook (.ipynb)	Python Script (.py)
Cách chạy	Chạy theo từng cell	Chạy toàn bộ chương trình
Hiển thị kết quả	Hiển thị ngay bên dưới cell	Hiển thị trên terminal
Hỗ trợ Markdown	Có	Không
Phù hợp	Phân tích dữ liệu và thử nghiệm	Chương trình hoàn chỉnh và Production code

→ File .py chạy xong là mất toàn bộ biến trong bộ nhớ, còn Jupyter Notebook sử dụng một Kernel chạy liên tục nên có thể giữ lại dữ liệu và biến giữa các lần thực thi cell, giúp việc phân tích dữ liệu và thử nghiệm code thuận tiện hơn.

VD: file ở VS Code

Search

Update

Welcomepandas_customerorderinfo.py 1customer_order.ipynb X

Users > ins > Documents > customer_order.ipynb > M4 Customer Order Analysis > M4 Visualization > import matplotlib.pyplot as plt

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.14.5

Customer Order Analysis

```
[25] ✓ 0.0simport pymysql
import pandas as pd
```

Database Connection

In this step, a connection to the ClassicModels database is established using PyMySQL.

```
[26] ✓ 0.0sconn = pymysql.connect(
    host="localhost",
    user="nhhy",
    password="Hoangyen9626!",
    database="classicmodels"
)
```

Load Customer Data

Retrieve customer number and customer name from the customers table.

```
[27] ✓ 0.0scustomers = pd.read_sql(
    "SELECT customerNumber, customerName FROM customers",
    conn
)
```

Ln 2, Col 22Spaces: 4Spaces: 4LFCell 17 of 20

Search

Update

Welcomepandas_customerorderinfo.py 1customer_order.ipynb X

Users > ins > Documents > customer_order.ipynb > M4 Customer Order Analysis > M4 Visualization > import matplotlib.pyplot as plt

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.14.5

Load Customer Data

Retrieve customer number and customer name from the customers table.

```
[27] ✓ 0.0scustomers = pd.read_sql(
    "SELECT customerNumber, customerName FROM customers",
    conn
)

customers.head()
```

... [/var/folders/v6/s8fn5ntn2ms_7tyvk6l48nf00000gn/T/ipykernel_71405/157101169.py:1](#): UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or customers = pd.read_sql()

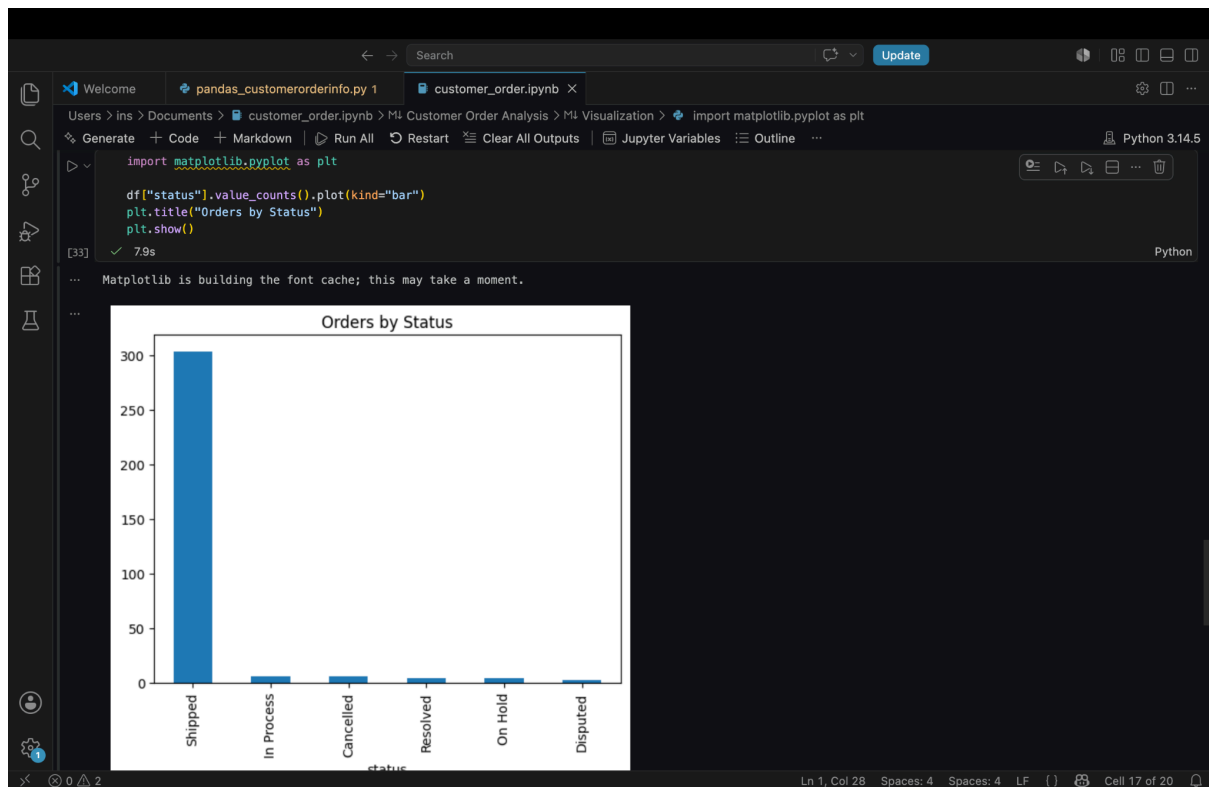
	customerNumber	customerName
0	103	Atelier graphique
1	112	Signal Gift Stores
2	114	Australian Collectors, Co.
3	119	La Rochelle Gifts
4	121	Baane Mini Imports

Dataset Inspection

The following checks are performed:

- Number of rows and columns
- Data types
- Summary statistics

Ln 2, Col 22Spaces: 4Spaces: 4LFCell 17 of 20



Link tham khảo:

<https://code.visualstudio.com/docs/datascience/jupyter-notebooks>

Kiến trúc Data Lakehouse

Khái niệm

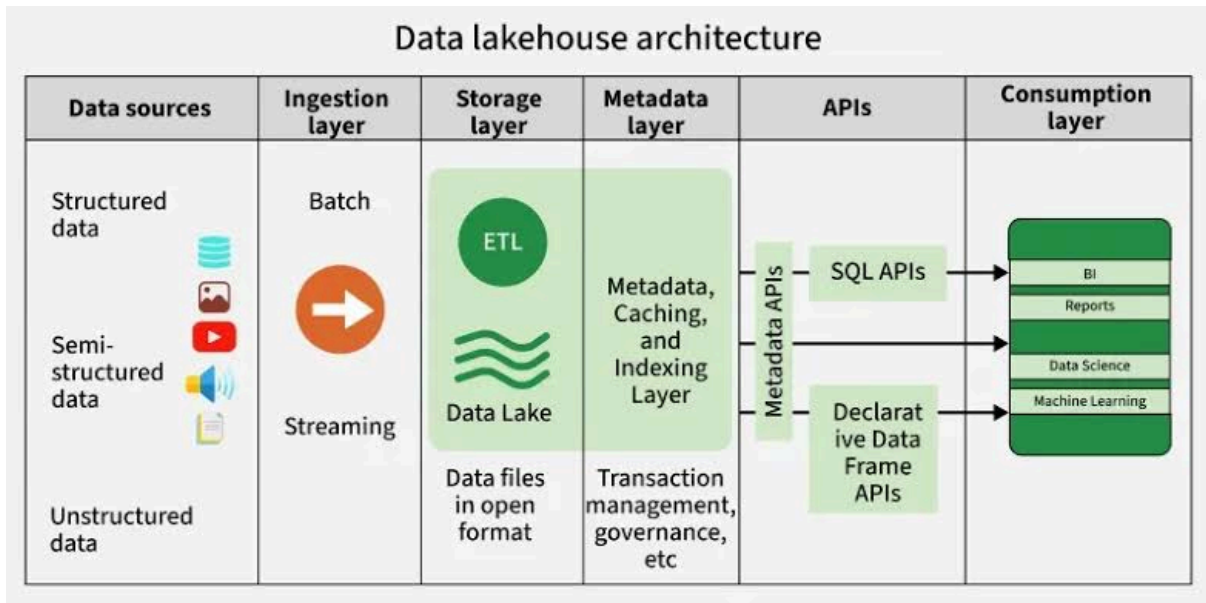
Data Lakehouse là một kiến trúc quản lý dữ liệu mở

- + Kết hợp tính linh hoạt, hiệu quả về chi phí và quy mô của các Data Lakes với việc quản lý dữ liệu và giao dịch ACID của Data Warehouse
- + Cho phép kích hoạt business intelligence (BI) và machine learning (ML) trên tất cả các kiểu dữ liệu.

Data Lakehouse hỗ trợ đọc và ghi dữ liệu đồng thời, truy cập trực tiếp vào dữ liệu nguồn, tách biệt tài nguyên lưu trữ và tính toán, tiêu chuẩn hóa các định dạng lưu trữ.

Kiến trúc

Kiến trúc data lakehouse bao gồm các thành phần chính:



1. Ingestion layer:

- Thu thập dữ liệu từ nhiều nguồn khác nhau theo thời gian thực:
- + Batch Processing: Nạp dữ liệu theo định kỳ (ví dụ hàng giờ, hàng ngày) từ các nguồn như SQL Server, Oracle.
- + Streaming & Real-time: Thu thập dữ liệu ngay lập tức từ các thiết bị IoT, log website hoặc ứng dụng di động thông qua Kafka.
- + Change Data Capture (CDC): Giúp đồng bộ các thay đổi nhỏ nhất từ cơ sở dữ liệu gốc vào Lakehouse mà không làm quá tải hệ thống nguồn.
- Fabric Data Factory (dịch vụ tích hợp dữ liệu của Microsoft Fabric) cho phép triển khai luồng dữ liệu và đường ống để thu thập, chuẩn bị và chuyển đổi dữ liệu trên nhiều nguồn khác nhau.

2. Storage layer:

- Lưu trữ khối lượng lớn dữ liệu với khả năng mở rộng cao và chi phí thấp.
- Thường sử dụng Cloud Object Storage (Amazon S3, Azure Blob Storage, OneLake).
- Dữ liệu được lưu trữ dưới dạng các tệp tin cột (Columnar files: Parquet/ORC), tối ưu hóa dung lượng nén và cho phép các công cụ tính toán chỉ đọc đúng những cột cần thiết.
- Lưu trữ dữ liệu ở mọi giai đoạn từ dữ liệu thô (Bronze), dữ liệu đã làm sạch (Silver) đến dữ liệu phục vụ phân tích (Gold).

3. Metadata layer:

- Cung cấp bối cảnh và cấu trúc của dữ liệu.
- Các công nghệ như Delta Lake, Apache Iceberg hoặc Hudi đóng vai trò là lớp Metadata trung gian. Cung cấp các tính năng: ACID, Time Travel, Schema Evolution

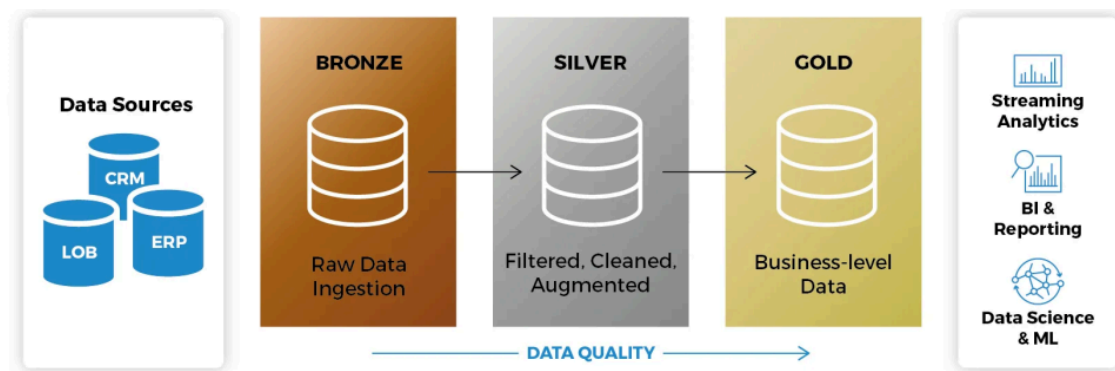
4. API/Compute & Query Engine Layer

- Cung cấp giao diện để truy cập và tương tác với dữ liệu.
- Chịu trách nhiệm xử lý, phân tích và truy vấn dữ liệu được lưu trữ trong Lakehouse; cung cấp các công cụ tính toán phân tán và thực thi SQL hiệu năng cao trên dữ liệu lớn.
- Các công nghệ phổ biến: Apache Spark, Spark SQL, SQL Endpoint,...
- Chức năng: ETL/ELT, Data Processing, Machine Learning,...

5. Consumption layer:

- Bao gồm các công cụ và nền tảng cho phép người dùng phân tích và trực quan hóa dữ liệu, bao gồm các công cụ BI như Power BI, hay Fabric Data Science, sử dụng dữ liệu được lưu trữ trong lakehouse.
- Biến dữ liệu thô thành những thông tin giá trị, hỗ trợ đưa ra quyết định dựa trên dữ liệu.

→ **Thực tế: Medallion Architecture**



Link tham khảo:

<https://www.microsoft.com/vi-vn/microsoft-fabric/resources/data-101/what-is-data-lakehouse>

<https://indaacademy.vn/dwh/kien-truc-data-lakehouse/>

Tính toán phân tán

Khái niệm & Nguyên lý hoạt động

Điện toán phân tán (Distributed Computing) là mô hình điều hành nhiều thiết bị độc lập (máy tính, server, node,...) cùng hợp tác để thực hiện một tác vụ hoặc giải quyết các vấn đề phức tạp.

→ Phân tán chia nhỏ dữ liệu hoặc tác vụ thành nhiều phần, phân phối đến các node khác nhau để xử lý song song, sau đó tổng hợp kết quả cuối cùng.

Phân biệt song song và phân tán:

- Tính toán song song: Sử dụng đồng thời nhiều bộ xử lý (processor) trên cùng một hệ thống.
- Tính toán phân tán: Sử dụng đồng thời nhiều máy tính (computer) độc lập qua mạng.

Các mô hình cơ bản

a. Client-server

Là mô hình phổ biến nhất, cho phép nhiều hệ thống làm việc đồng thời. Trong mô hình này, client (máy khách) gửi yêu cầu đến server (máy chủ). Server tiếp nhận yêu cầu, xử lý tác vụ hoặc phân bổ tài nguyên, sau đó phản hồi kết quả cho client.

b. Kiến trúc ba bậc (Three-tier)

Kiến trúc three-tier chia hệ thống thành ba lớp:

- Lớp giao diện (Presentation Tier): Giao diện người dùng.
- Lớp xử lý ứng dụng (Application Tier): Kiểm soát chức năng của ứng dụng
- Lớp dữ liệu (Data Tier): Lưu trữ dữ liệu.

Dữ liệu được lưu trữ tập trung tại Data Tier, tách biệt khỏi tầng giao diện và tầng xử lý nghiệp vụ.

c. Kiến trúc ngang hàng

Các node hoạt động đồng đẳng, mỗi node (nút) có thể vừa đóng vai trò là client vừa là server, không có node trung tâm.

d. Kiến trúc N bậc

Mở rộng mô hình three-tier, không giới hạn số lượng tầng, cho phép bổ sung thêm các tầng chức năng tùy thuộc vào yêu cầu của ứng dụng. Cách tổ chức tương tự như three-tier nhưng linh hoạt hơn và thường ứng dụng làm cơ sở kiến trúc cho các dịch vụ web và hệ thống dữ liệu lớn.

Ưu điểm

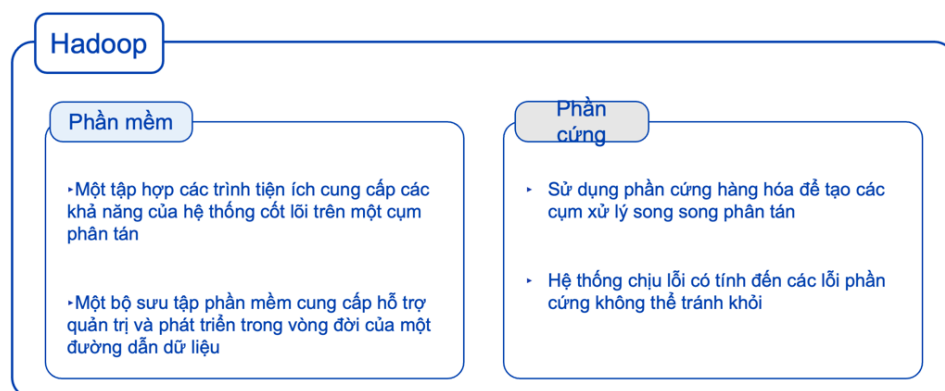
- Khả năng mở rộng (Scalability)
- Hiệu năng cao hơn
- Tăng tính sẵn sàng (Availability)
- Chịu lỗi tốt (Fault Tolerance)

Link tham khảo:

Hadoop Ecosystem

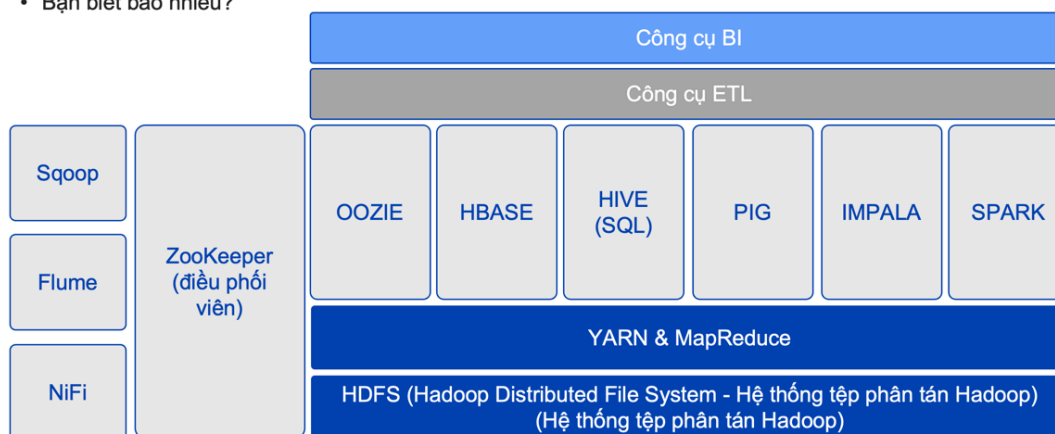
Hadoop Ecosystem là tập hợp các công cụ hỗ trợ lưu trữ, xử lý, quản lý và phân tích dữ liệu lớn trên môi trường phân tán.

- Xử lý dữ liệu nơi nó được lưu trữ (Data Locality)
 - Sử dụng daemon (chương trình chạy nền: NameNode, DataNode, ResourceManager, NodeManager) dựa trên phần mềm thay vì phần cứng để giải quyết các thách thức của điện toán phân tán
 - Kiến trúc Master-Slave nơi các nô lệ có thể được thêm vào để mở rộng nền tảng
 - Cung cấp một nền tảng nơi các nhà phát triển và người dùng có thể tập trung vào việc xử lý dữ liệu mà không phải lo lắng về các nhiệm vụ quản lý liên quan đến nền tảng
- Tập hợp phần mềm và phần cứng để xử lý Big Data

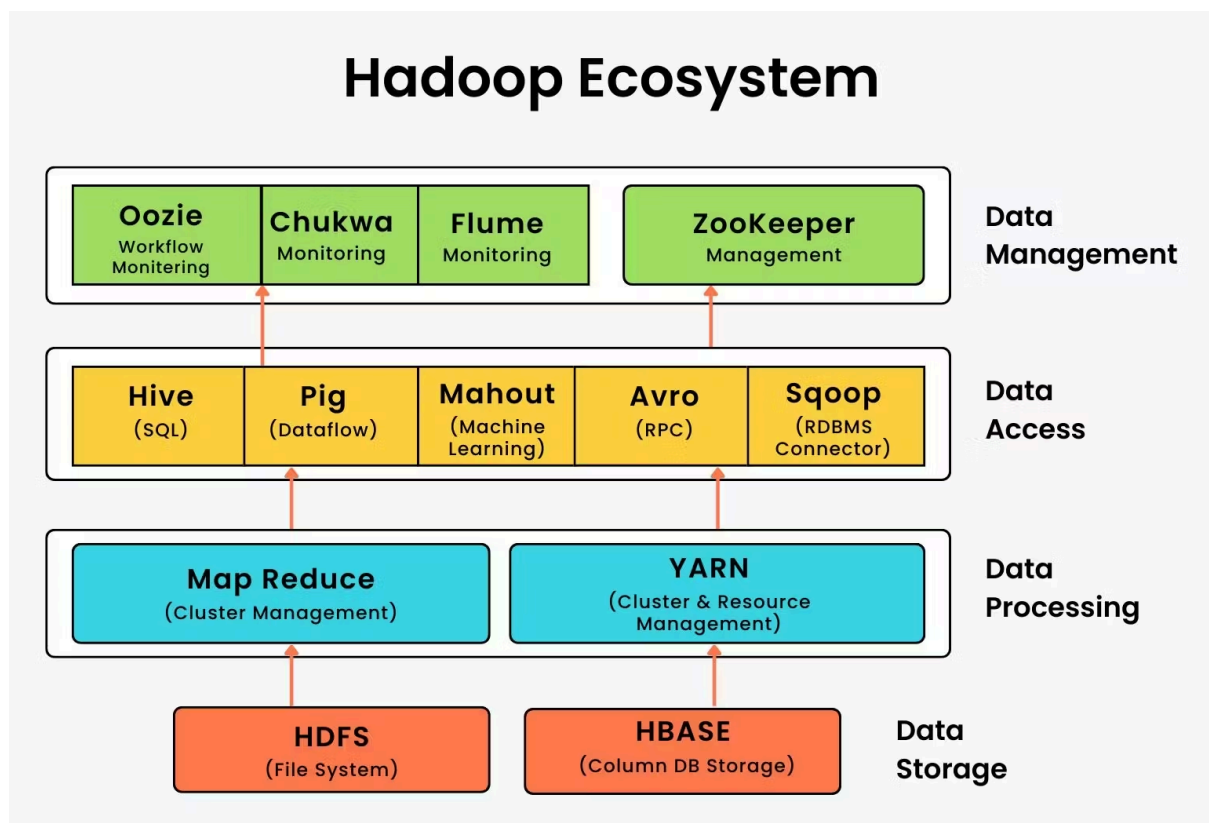
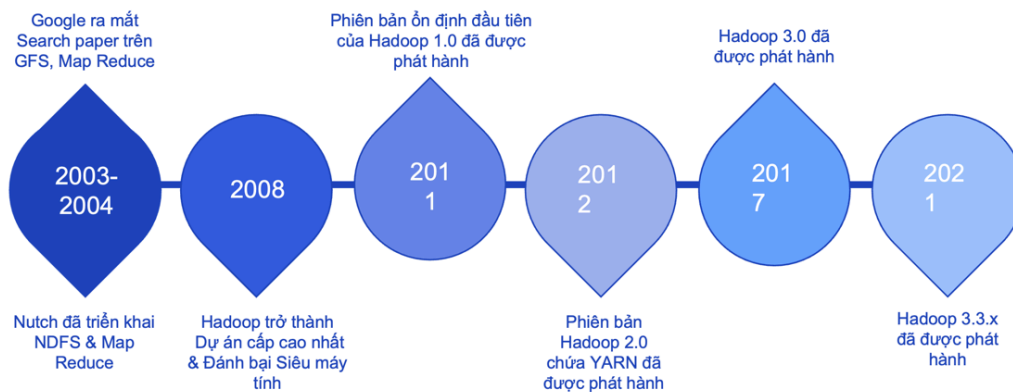


Hệ sinh thái Hadoop

- Bạn biết bao nhiêu?



Nguồn gốc của Hadoop



1. Data Storage Layer

Là tầng lưu trữ dữ liệu của Hadoop, gồm có

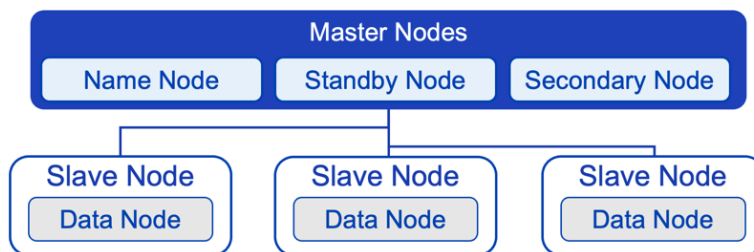
a. HDFS (Hadoop Distributed File System)

HDFS: hệ thống tệp phân tán của Hadoop, được thiết kế để lưu trữ dữ liệu rất lớn trên nhiều máy chủ.

- Chia file thành nhiều block (thường 128MB hoặc lớn hơn).
- Phân phối các block lên nhiều node.
- Nơi lưu trữ dữ liệu gốc cho toàn bộ hệ sinh thái Hadoop.
- Tuân theo kiến trúc Master - Slave.

Thành phần HDFS

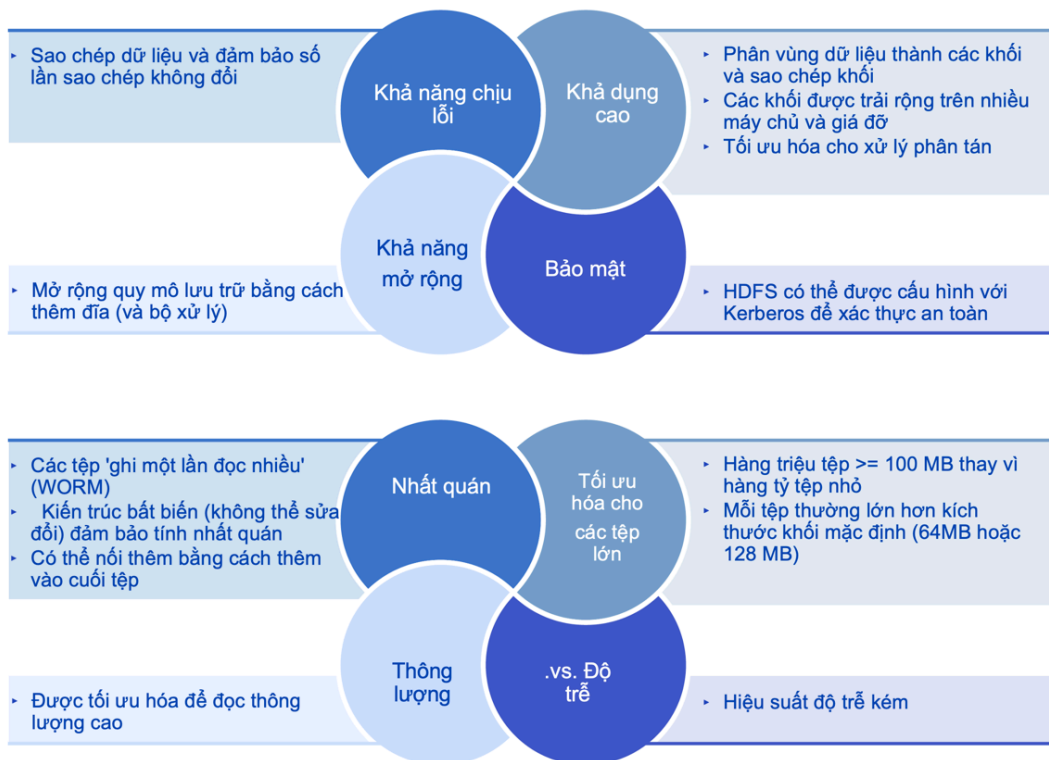
- Name Node
 - Trình nền chủ đang hoạt động
 - Xác định và duy trì cách các khối dữ liệu được phân phối trên các DataNodes
 - Không tham gia vào hoạt động đọc/ghi thực tế
- Secondary / Standby Node
 - Trình nền chủ thụ động
 - Chịu trách nhiệm duy trì độ bền, siêu dữ liệu mà Name Node lưu trữ trong bộ nhớ + thay thế Name Node nếu nó bị lỗi
- DataNode
 - Đọc và ghi các khối dữ liệu
 - Chịu trách nhiệm sao chép các khối trên các DataNode khác



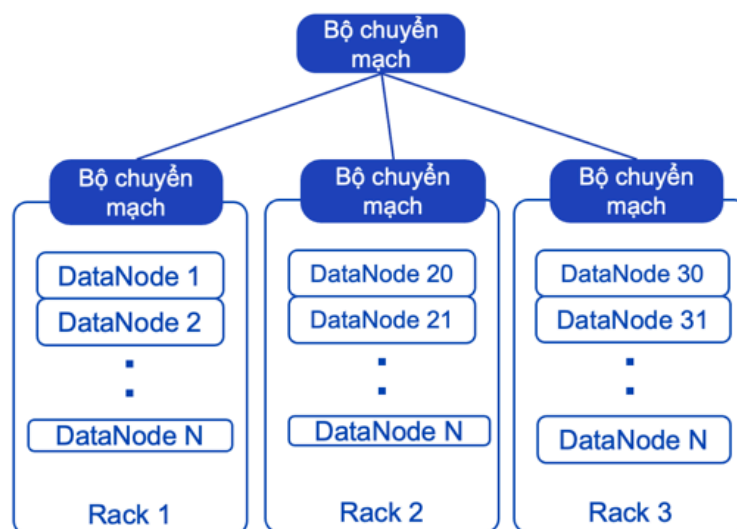
HDFS hoạt động trên hệ thống tệp hệ thống (thường là Linux)

- Mô phỏng hệ thống tệp Linux với các tệp và thư mục
- Dựa trên Hệ thống tệp của Google (GFS)
- Sử dụng phần cứng tiêu chuẩn công nghiệp có thể mở rộng
- Dữ liệu được phân vùng thành các khối và phân phối trên nhiều máy chủ trong quá trình ghi.





- Ít block hơn $\Rightarrow$ ít metadata hơn $\Rightarrow$ NameNode đỡ phải quản lý hơn.
 - Tất cả các khối trong một tệp có cùng kích thước, ngoại trừ khối cuối cùng
- HDFS có thể được cấu hình để nhận biết cấu trúc liên kết giá cụm và các máy chủ “gần” với nhau như thế nào
- Closest: sẽ ở trên cùng một máy chủ
 - Closer: Trên cùng một giá đỡ
- Máy khách đọc các khối dữ liệu từ máy chủ "gần nhất" bất cứ khi nào có thể. HDFS sử dụng cấu trúc liên kết giá đỡ trong các thao tác ghi để tăng khả năng chịu lỗi.



Khả năng mở rộng và chịu lỗi của HDFS

- Khả năng mở rộng (Scalability)

- + HDFS mở rộng theo chiều ngang (Horizontal Scaling).
- + Khi dung lượng lưu trữ/khối lượng dữ liệu tăng lên, hệ thống có thể bổ sung thêm các DataNode mới vào cluster mà không cần thay đổi kiến trúc hiện có.
- Khả năng chịu lỗi (Fault Tolerance)
- + HDFS đảm bảo khả năng chịu lỗi thông qua cơ chế Replication.
- + Mỗi block dữ liệu thường được sao chép thành 3 bản trên các DataNode khác nhau.

Nếu một DataNode bị lỗi:

- NameNode phát hiện node mất kết nối
- Các bản sao còn lại vẫn được sử dụng
- HDFS tự động tạo thêm bản sao mới trên node khác

→ Hệ thống vẫn tiếp tục hoạt động mà không làm mất dữ liệu.

Hệ số sao chép mặc định là 3

- Tăng tính khả dụng - 3 vị trí để bắt đầu đọc
- Tăng độ tin cậy – Có thể bị tổn thất 66% và vẫn phục hồi

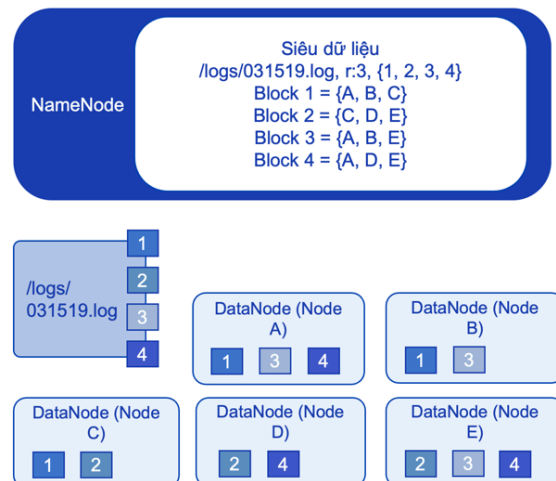
Vị trí của các khối được sao chép, ảnh hưởng lớn đến mức độ chịu lỗi

Tất cả các khối trên cùng một đĩa – thực sự không tăng độ tin cậy

Tất cả các khối trên cùng một giá – lỗi công tắc giá có thể khiến tất cả các bản sao không thể truy cập được

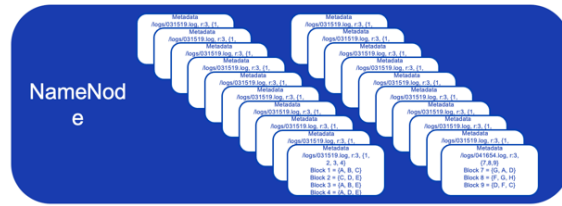
HDFS NameNode

- NameNode chịu trách nhiệm giữ siêu dữ liệu namespace
 - Siêu dữ liệu namespace bao gồm liên kết giữa các tệp, đó là các khối và vị trí của các khối đó
 - Siêu dữ liệu được lưu trữ trong bộ nhớ để có hiệu suất nhanh hơn
- Siêu dữ liệu cũng được lưu trữ trên đĩa để đảm bảo độ bền
 - Được lưu trữ trong tệp fsimage
 - fsimage không lưu trữ vị trí khối
 - Các thay đổi đối với siêu dữ liệu được ghi vào tệp nhật ký chỉnh sửa



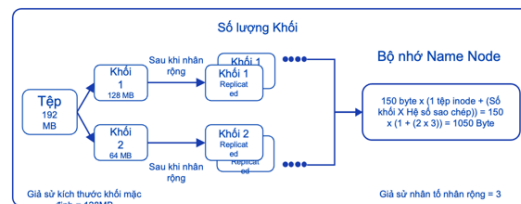
Cấp phát bộ nhớ NameNode

- Khi NameNode đang chạy, tất cả siêu dữ liệu được giữ trong phần hồi nhanh của RAM
- NameNode có Kích thước Heap Java mặc định là 1 GB
- NameNode lưu trữ siêu dữ liệu sau
 - Thông tin tệp - tên tệp, quyền sở hữu, quyền, v.v.
 - Thông tin khối - đối với mỗi khối là một phần của tệp, tên khối và vị trí
 - Mỗi mục sử dụng từ 150 đến 250 byte bộ nhớ

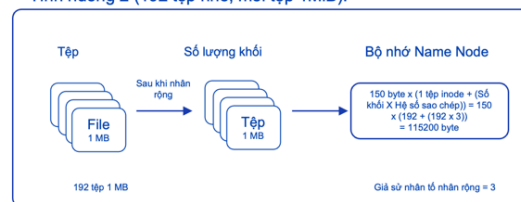


Sự cố tập nhỏ NameNode

- HDFS bị vấn đề về tệp nhỏ
- Đây là khi một phần lớn các tệp trong HDFS là các tệp nhỏ
 - HDFS NameNode có thể hết bộ nhớ Java trước khi hết dung lượng đĩa
- Ví dụ: 1 GB dữ liệu được lưu trữ dưới dạng một tệp .vs. 1000 tệp 1 MB
- Tệp 1 GB duy nhất có kích thước khối là 128 MB
 - 1 Thông tin tệp và 8 Thông tin khối
 - Tổng cộng 8 mục trong bộ nhớ
- Tệp 1000 x 1 MB với kích thước khối là 128 MB
 - Thông tin 1000 tệp và 1000 thông tin khối



Tình huống 2 (192 tệp nhỏ, mỗi tệp 1MiB):

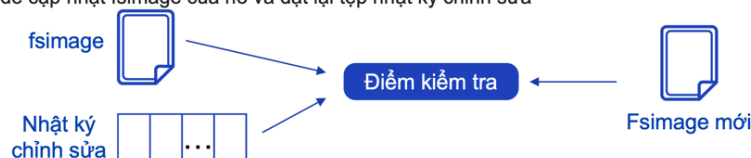


- Tổng số 2000 mục trong bộ nhớ

→ do lưu quá nhiều metadata

HDFS Secondary NameNode

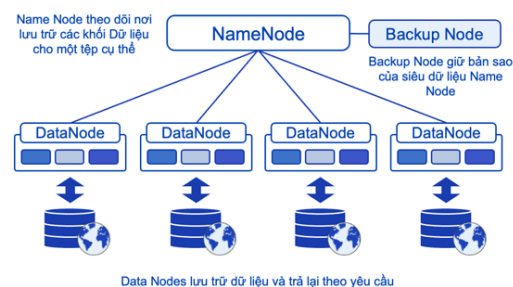
- NameNode phụ, mặc dù đúng với tên của nó, nhưng không phải là daemon chuyển đổi dự phòng cho NameNode
- Vai trò chính của nó là kiểm tra siêu dữ liệu không gian tên
 - Nhận một bản sao của fsimage và chỉnh sửa tệp nhật ký mới nhất từ NameNode
 - Hợp nhất các thay đổi trong tệp nhật ký chỉnh sửa vào fsimage
 - Gửi bản sao mới của fsimage tới NameNode
 - NameNode cập nhật fsimage của nó và đặt lại tệp nhật ký chỉnh sửa



Secondary NameNode không phải là NameNode dự phòng. Vai trò chính của nó là thực hiện checkpoint metadata. Nó định kỳ lấy fsimage và edit log từ NameNode, hợp nhất các thay đổi trong edit log vào fsimage để tạo ra fsimage mới, sau đó gửi lại cho NameNode. Việc này giúp edit log không bị quá lớn và giúp NameNode khởi động nhanh hơn khi restart.

HDFS DataNodes

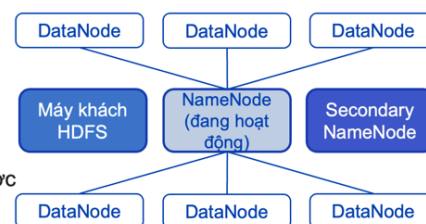
- DataNode là những công nhân thực tế chịu trách nhiệm viết và đọc các khối dữ liệu
- Khi viết các khối, các DataNode giao tiếp với nhau để sao chép khối theo hệ số sao chép
 - Các khối được sao chép theo kiểu đường ống
 - NameNode không phải là một phần của quá trình ghi.
 - Vai trò duy nhất của nó là ánh xạ các khối tới các nút dữ liệu
- Các khối được lưu dưới dạng tệp trong hệ thống tệp cơ bản
 - Vị trí lưu khối có thể được định cấu hình
 - Các khối được đặt tên là blk_xxxxxxxx
- DataNodes không biết khối thuộc về tệp nào



Chương 2. Những nguyên tắc cơ bản của Big Data

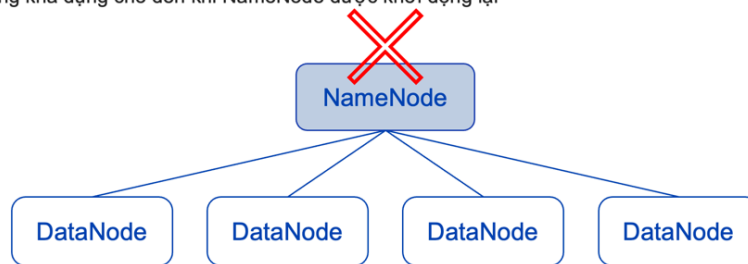
HDFS ở Chế độ không có tính khả dụng cao

- HDFS có thể được triển khai ở chế độ High Availability (HA) hay không
- HDFS có thể được triển khai ở chế độ High Availability (HA) hay không
 - NameNode (Chủ)
 - Secondary NameNode (Chủ)
 - DataNode (Tớ)
- Ở chế độ không HA, nếu NameNode bị lỗi, hệ thống phải được khởi động lại với NameNode phụ thay thế cho NameNode
 - Khi khởi động lại, NameNode mới đọc tệp fsimage
 - Tất cả các datanode gửi thông tin vị trí khối đến NameNode

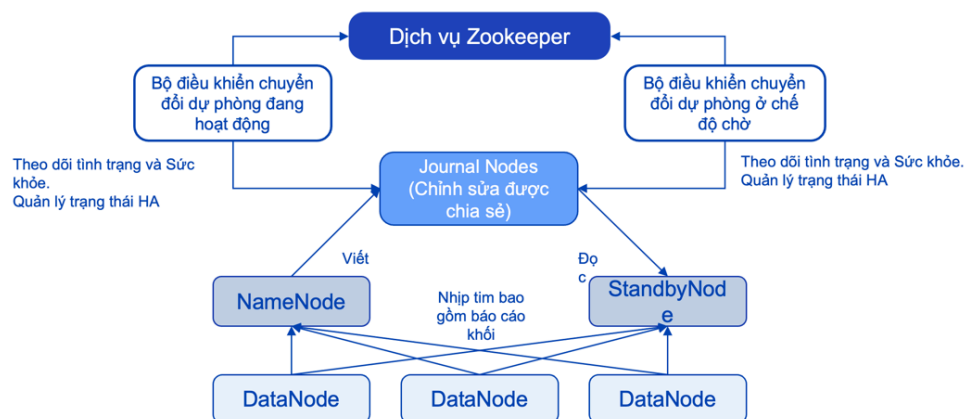


Single Point of Failure (SPOF): Điểm lỗi duy nhất

- Một cụm Hadoop có một NameNode đang hoạt động
 - Nếu tại bất kỳ thời điểm nào, thực sự có nhiều namenode đang hoạt động, thì cụm đó sẽ bị chia tách não
- Nếu NameNode đang hoạt động bị lỗi, toàn bộ dịch vụ HDFS sẽ không thể truy cập được
 - Đây là một điểm thất bại duy nhất
 - HDFS sẽ không khả dụng cho đến khi NameNode được khởi động lại



HDFS in High Availability Mode



mọi thay đổi metadata đều ghi vào journal nodes

- Secondary NameNode = checkpoint metadata.
- Standby NameNode = NameNode dự phòng thực sự để High Availability.

DataNode Thất bại và phục hồi

- DataNode gửi nhịp tim đến NameNode theo định kỳ
 - Tần suất có thể được cấu hình với mặc định là 3 giây
- Nếu nhịp tim không được nhận từ DataNode, nó sẽ tiến triển qua nhiều giai đoạn và cuối cùng được tuyên bố là đã chết
 - Ban đầu được tuyên bố là cũ sau 30 giây
 - Được tuyên bố là đã chết sau 10,5 phút
- NameNode loại trừ các DataNode cũ tham gia vào quá trình ghi mới
- Sau khi được tuyên bố là đã chết, tất cả các khối trên DataNode đã chết được coi là bị mất
 - Điều này sẽ kích hoạt quy trình khôi phục chưa được sao chép và các khối trên nút dữ liệu chết sẽ được sao chép lại trên các nút dữ liệu khác
 - Nếu một DataNode tham gia lại cụm sau một thời gian chết, NameNode đảm bảo rằng các khối đó không bị sao chép quá mức bằng cách xóa chúng

Truy cập HDFS từ Dòng lệnh

Từ dòng lệnh, sử dụng `hdfs dfs -<lệnh phụ> <tùy chọn> <tham số>`

Danh sách các lệnh con thường được sử dụng

- `ls` – liệt kê nội dung của một đạo diễn hdfs
- `cp` – sao chép tệp hdfs
- `mv` – di chuyển tệp hdfs
- `mkdir` – tạo một thư mục hdfs
- `put` - sao chép tệp Linux cục bộ sang hdfs
- `get` – sao chép tệp hdfs sang tệp Linux

VD:

Input: `hdfs dfs -ls /`

Output:

`drwxr-xr-x /data`

`drwxr-xr-x /logs`

`drwxr-xr-x /raw`

Các lệnh Shell của hệ thống tập tin HDFS

Lệnh	Mô tả
ls	liệt kê nội dung của các thư mục
du	hiển thị việc sử dụng đĩa
count	đếm số lượng thư mục, tệp và byte trong một đường dẫn
chgrp, chown, chmod	thay đổi quyền truy cập tập tin và thư mục
stat	in số liệu thống kê về một tập tin hoặc thư mục
cat, text	hiển thị nội dung của các tập tin
tail	hiển thị 1KB cuối cùng của nội dung tệp
get, copyToLocal	các lệnh giống hệt nhau sao chép tệp từ HDFS sang hệ thống tệp cục bộ
put, copyFromLocal	các lệnh giống hệt nhau sao chép tệp từ hệ thống tệp cục bộ vào HDFS
getmerge	lấy một tập hợp các tệp và hợp nhất chúng thành một tệp duy nhất

Lệnh	Mô tả
mv	di chuyển một tệp trong HDFS
cp	sao chép tệp sang vị trí khác trong HDFS
mkdir	tạo một thư mục mới trong HDFS
rm	xóa một tệp (vào thư mục Thùng rác)
rm -R	xóa các thư mục và xóa đệ quy mọi tệp và thư mục con (vào thư mục Thùng rác)
test	kiểm tra nếu một tập tin tồn tại
touch	ghi dấu thời gian vào một tệp mới
expunge	làm trống thư mục Thùng rác của người dùng

b. HBase

Là cơ sở dữ liệu NoSQL dạng cột được xây dựng trên HDFS.

- Truy cập dữ liệu thời gian thực.
- Hỗ trợ đọc ghi ngẫu nhiên nhanh.
- Phù hợp với dữ liệu cực lớn.
- Lưu trữ dữ liệu có cấu trúc cần truy vấn nhanh

→ Mọi cluster Hadoop đều có HDFS, nhưng không phải cluster Hadoop nào cũng có HBase.

2. Data Processing Layer

Là tầng xử lý dữ liệu, gồm có:

a. MapReduce

Là framework xử lý dữ liệu phân tán của Hadoop cho Big Data, gồm 2 bước:

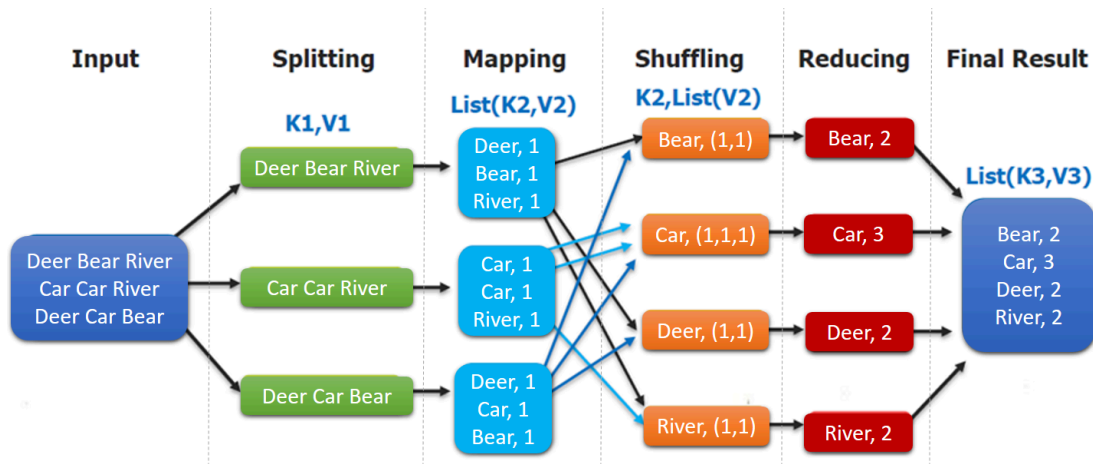
- Map: Chuyển đổi dữ liệu đầu vào thành các cặp Key-Value.
- Reduce: Tổng hợp các giá trị có cùng Key để tạo ra kết quả cuối cùng.

Ưu điểm:

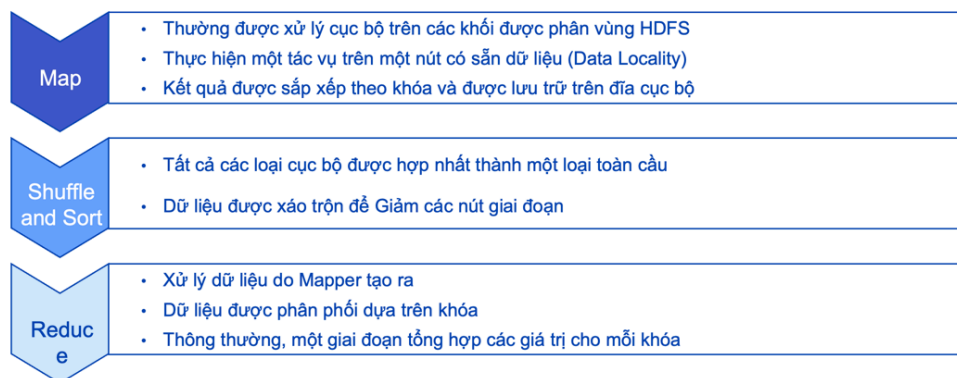
- Tự động song song hóa và phân phối
- Khả năng chịu lỗi
- Công cụ giám sát và trạng thái (running/completed)
- Thân thiện với lập trình viên

Cơ chế hoạt động

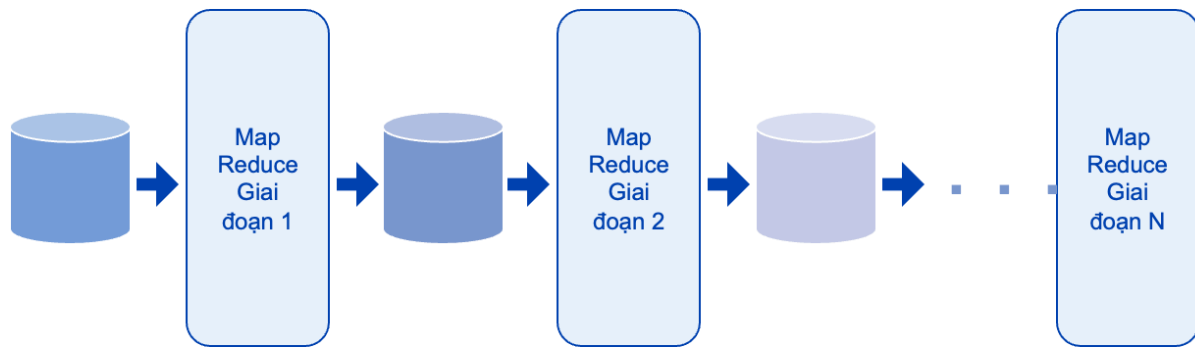
Chia dữ liệu đầu vào thành nhiều khối (Input Split) → Thực hiện các tác vụ Map trên từng khối dữ liệu → Sinh ra các cặp Key-Value trung gian → Thực hiện Shuffle & Sort để gom các bản ghi có cùng Key → Reduce xử lý từng nhóm Key → Ghi kết quả ra hệ thống lưu trữ



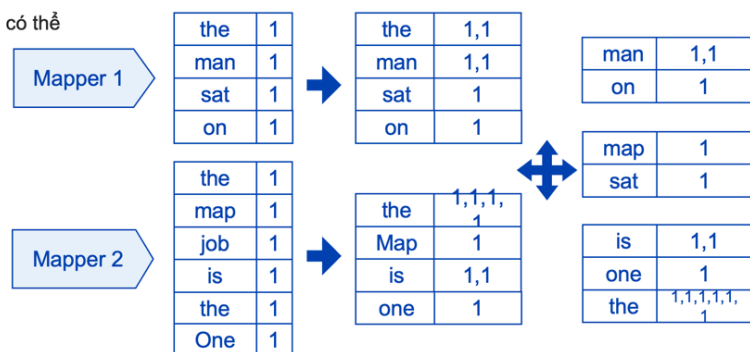
Ba giai đoạn của MapReduce



- Mappers đọc dữ liệu từ đĩa và thực hiện tính toán
- Sau khi tất cả những người lập bản đồ đã hoàn thành, dữ liệu được xáo trộn, sắp xếp và ghi vào đĩa
- Các Reducers đọc dữ liệu, thực hiện rút gọn và ghi kết quả vào đĩa



- Giai đoạn Shuffle Sort
 - Kết quả của mỗi Mapper được sắp xếp theo khóa và được lưu trữ trên đĩa cục bộ
 - Theo tùy chọn, để có hiệu suất tốt hơn, khi các phim được sắp xếp, chúng có thể được tổng hợp



Bottleneck

Bước reduce chỉ có thể tiến hành sau khi tất cả những mapper đã hoàn thành

Mapper chậm nhất quyết định tốc độ

VD: Word count VS Code

Điện toán đám mây (Cloud Computing)

Đang phát triển	Giai đoạn	Đặc trưng
	Điện toán lưới	Giải quyết các vấn đề lớn với điện toán song song Được Liên minh toàn cầu chủ đạo
	Điện toán tiện ích	Tài nguyên máy tính được cung cấp dưới dạng dịch vụ có đồng hồ đo vào cuối những năm 1990
	Phần mềm dưới dạng Dịch vụ	Phần mềm dựa trên đăng ký được truy cập qua Internet đã đạt được động lực sau năm 2001
	Điện toán đám mây	Trung tâm dữ liệu thế hệ tiếp theo với công nghệ ảo hóa toàn bộ dịch vụ - IaaS, PaaS, & SaaS

Các đặc điểm chính:

- Dịch vụ tự phục vụ theo yêu cầu

- Truy cập mạng phổ biến
- Tập hợp tài nguyên không phụ thuộc vào vị trí
- Tính đàn hồi nhanh
- Thanh toán cho mỗi lần sử dụng

Dịch vụ đám mây và mô hình tiêu thụ

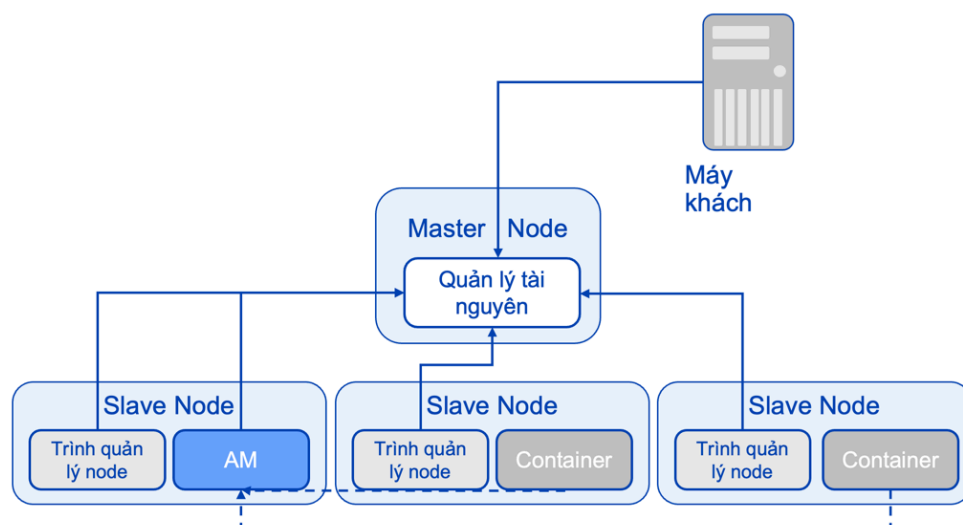
- So sánh SaaS, PaaS và IaaS

Mô hình dịch vụ	Mô hình tiêu thụ	Mô tả
Software as a Service (SaaS) (Phần mềm dưới dạng dịch vụ)	Trực tiếp tiêu dùng dịch vụ	Ứng dụng người dùng cuối được phân phối dưới dạng dịch vụ
Platform as a Service (PaaS) (Nền tảng dưới dạng Dịch vụ)	Phát triển và xây dựng trên nền tảng	Nền tảng ứng dụng hoặc phần mềm trung gian được cung cấp dưới dạng dịch vụ
Infrastructure as a Service (IaaS) (Cơ sở hạ tầng như một dịch vụ)	Cơ sở hạ tầng theo yêu cầu	Máy tính, lưu trữ hoặc cơ sở hạ tầng CNTT khác được cung cấp dưới dạng dịch vụ

Trong lý thuyết, dữ liệu được đọc song song tốt nhất khi mỗi block nằm trên một node riêng. Tuy nhiên trong thực tế HDFS phải cân bằng nhiều yếu tố như replication, dung lượng lưu trữ và số lượng node nên các block không bao giờ được phân phối hoàn hảo. Vì vậy một số node có thể phải xử lý nhiều block hơn node khác, làm giảm mức độ song song tối đa của hệ thống.

b. YARN (Yet Another Resource Negotiator)

Là thành phần quản lý tài nguyên và lập lịch thực thi trong Hadoop, giúp phân phối CPU, RAM và các tài nguyên khác cho các ứng dụng đang chạy trong cluster. Kiến trúc Master/Slave.

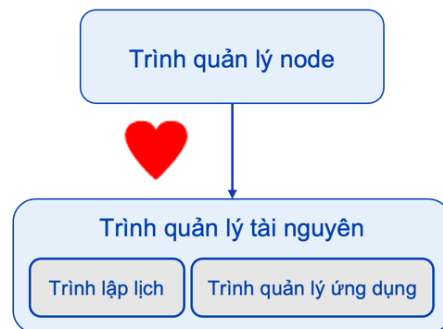


Các thành phần chính:

- ResourceManager: Quản lý tài nguyên toàn cluster.
- + Scheduler: Trình lập lịch.
- + Application Manager: Trình quản lý ứng dụng.
- NodeManager: Quản lý tài nguyên từng node.
- ApplicationMaster: Điều phối một ứng dụng cụ thể.
- Container: Đơn vị tài nguyên được cấp phát.

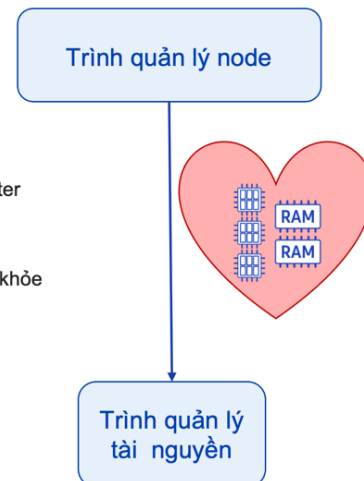
Trình quản lý tài nguyên (RM) YARN

- Trình nền master
- Hai thành phần chính: Scheduler và ApplicationsManager
- Scheduler (Trình lập lịch):
 - Chịu trách nhiệm phân bổ tài nguyên cho các ứng dụng đang chạy khác nhau
 - Chính sách có thể cấm – CapacityScheduler, FairScheduler
- ApplicationsManager (Trình quản lý ứng dụng)
 - Chịu trách nhiệm tiếp nhận hồ sơ công việc
 - Đảm bảo phân vùng chứa cho ApplicationMaster trên mỗi ứng dụng và theo dõi nhịp tim của nó
- Theo dõi nhịp tim từ Trình quản lý node



Trình quản lý node YARN

- Trình nền worker
- Chạy trên worker node và thường cùng với DataNode
- Chịu trách nhiệm khởi chạy và quản lý các thùng chứa cho ứng dụng
 - Các bộ chứa thực thi các tác vụ theo chỉ định của mỗi ứng dụng AppMaster
- Theo dõi sức khỏe và tài nguyên của node mà nó đang chạy trên đó
 - Gửi nhịp tim tới Trình quản lý tài nguyên với trạng thái tài nguyên và sức khỏe của node hiện tại



Khả năng chịu lỗi của YARN

- Có nhiều trình nền liên quan đến việc chạy một ứng dụng
- YARN xử lý các trường hợp ngoại lệ như thế nào?
- Các tác vụ trên Trình quản lý node thoát với ngoại lệ hoặc ngừng gửi nhịp tim
 - ApplicationMaster thử lại tác vụ trên một vùng chứa mới trên một nút khác
 - Thử lại tối đa 4 lần
- ApplicationMaster ngừng gửi nhịp tim tới YARN
 - Trình quản lý tài nguyên bắt đầu một ApplicationMaster mới và thử lại toàn bộ ứng dụng
 - Thử lại tối đa 2 lần
- Trình quản lý tài nguyên chết
 - Có thể chạy ResourceManager ở chế độ HA
 - Ở chế độ HA, Trình quản lý tài nguyên dự phòng sẽ tự động chuyển đổi dự phòng

Dòng lệnh YARN

Cú pháp cơ bản: yarn <lệnh con> <args>

- yarn application –list

Để xem tất cả các ứng dụng do YARN quản lý hiện đang chạy trên cụm.

- yarn application –status <app-id>

Để hiển thị trạng thái của một ứng dụng riêng lẻ.

- yarn application –kill <app-id>

Để tắt một ứng dụng đang chạy được xác định bởi id ứng dụng.

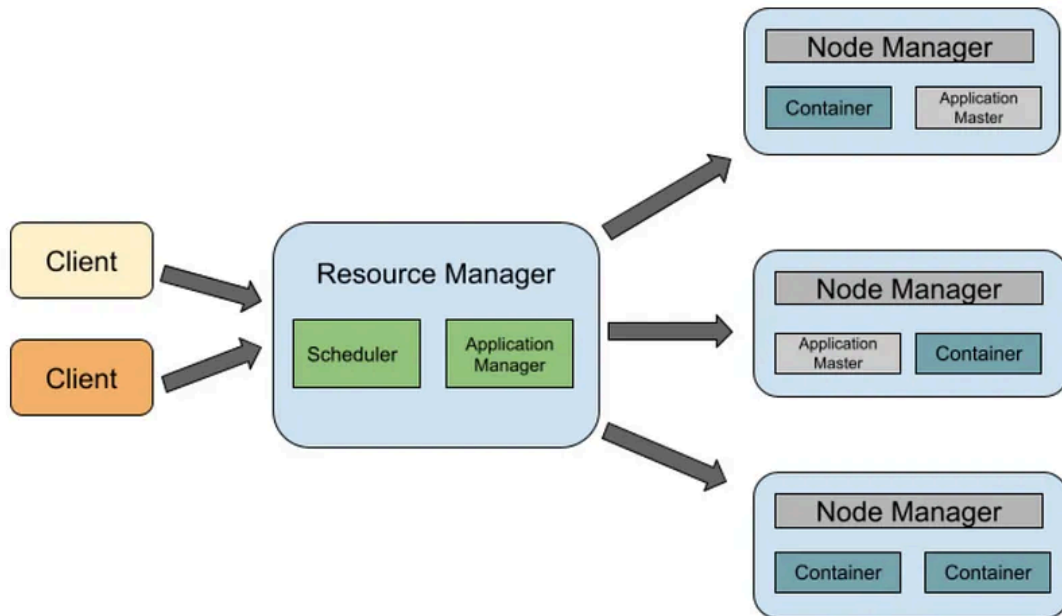
Cơ chế phân chia tài nguyên của YARN

Client gửi job lên cluster → ResourceManager tạo ApplicationMaster

→ ApplicationMaster yêu cầu tài nguyên → ResourceManager cấp các Container

→ NodeManager khởi chạy Container → Các task được thực thi trong Container

→ Kết quả trả về cho người dùng.



Cơ chế quản lý RAM của YARN

YARN quản lý tài nguyên dựa trên:

- Memory (RAM)
- CPU (vCore)

Mỗi Container được cấp một lượng RAM xác định. NodeManager liên tục theo dõi mức sử dụng bộ nhớ của Container. Nếu Container vượt quá giới hạn được cấp phát, YARN có thể dừng Container đó để bảo vệ tài nguyên của cluster.

Cơ chế ghi Log trong YARN

Mỗi ứng dụng khi chạy sẽ sinh log tại NodeManager. Các loại log phổ biến: stdout, stderr, syslog,...

Log được lưu lại để:

- Theo dõi tiến trình chạy.
- Kiểm tra lỗi.
- Phân tích hiệu năng.

Sau khi job hoàn thành, log có thể được gom và lưu tập trung để phục vụ việc giám sát hệ thống.

Có thể xem log bằng lệnh **yarn logs -applicationId <app-id>**.

Spill Memory trong Hadoop/Spark

- Spill xảy ra khi dữ liệu trung gian lớn hơn bộ nhớ được cấp phát.
- Quy trình: Dữ liệu được xử lý trong RAM → Bộ nhớ đầy → Dữ liệu trung gian được ghi tạm xuống đĩa (spill) → Tiếp tục xử lý và merge kết quả.
- Spill giúp tránh lỗi Out Of Memory nhưng làm giảm hiệu năng do phải đọc/ghi đĩa.

3. Data Access Layer

Là tầng cho phép người dùng truy cập và khai thác dữ liệu.

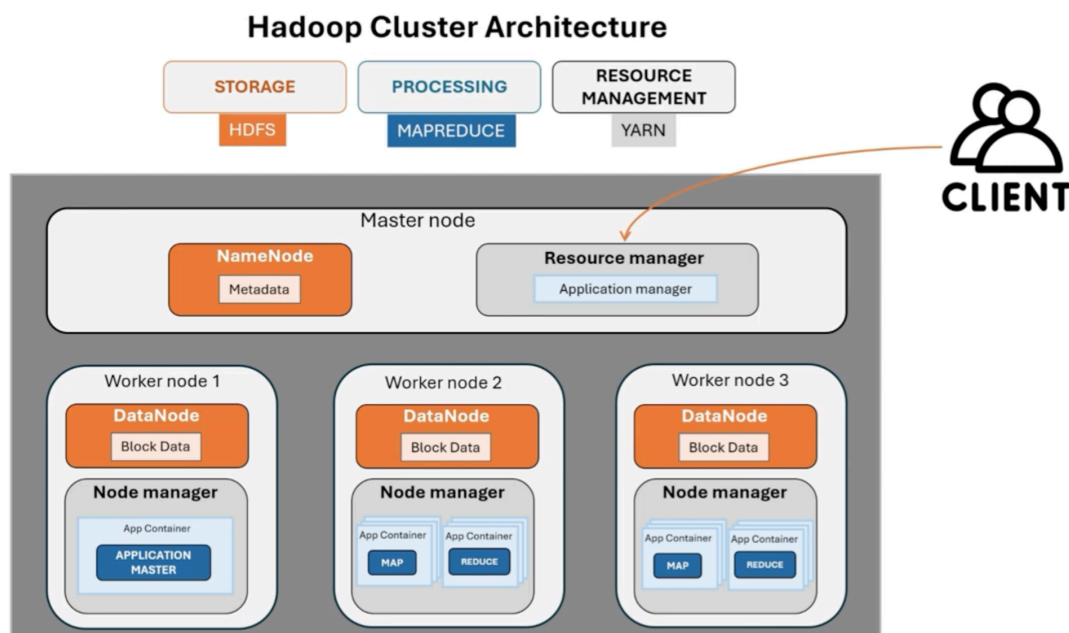
- Hive: Data Warehouse trên Hadoop, cho phép dùng ngôn ngữ gần giống SQL.
- Pig: Cung cấp ngôn ngữ Pig Latin để xử lý dữ liệu, giúp viết pipeline xử lý dữ liệu ngắn hơn MapReduce thuần.
- Mahout: Thư viện Machine Learning cho Hadoop. Các thuật toán: Clustering, Classification, Recommendation.
- Avro: Framework tuần tự hóa dữ liệu (Data Serialization), giúp trao đổi dữ liệu giữa các hệ thống, giảm kích thước dữ liệu và hỗ trợ schema evolution.
- Sqoop: Trao đổi dữ liệu giữa Hadoop và cơ sở dữ liệu quan hệ.

4. Data Management Layer

Tầng quản trị và vận hành hệ thống.

- Oozie: Workflow scheduler cho Hadoop, đảm bảo các job chạy đúng thứ tự.
- Chukwa: Hệ thống thu thập và giám sát log được xây dựng trên Hadoop.
- Flume: Công cụ thu thập dữ liệu log và sự kiện theo thời gian thực
- ZooKeeper: Dịch vụ điều phối phân tán giúp quản lý cấu hình và đồng bộ hóa giữa các node.

Trong thực tế: Core Hadoop bao gồm HDFS, MapReduce và YARN



Link tham khảo:

<https://viblo.asia/p/gioi-thieu-tong-quan-ve-hadoop-va-hadoop-ecosystems-m2vJPog84eK>

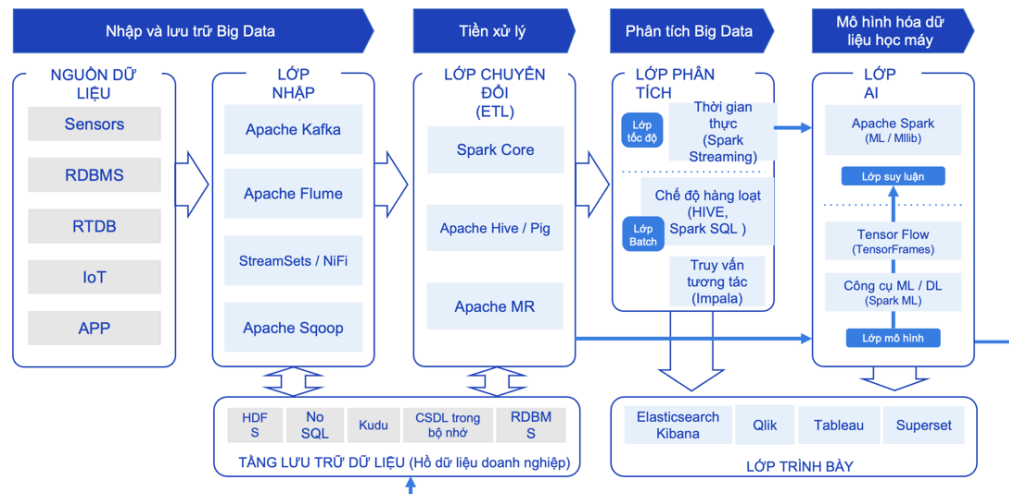
<https://www.edureka.co/blog/hadoop-yarn-tutorial/>

Big Data

Ứng dụng và xử lý Big Data

Mục tiêu phân tích Big Data là dịch chuyển từ Phân tích dự đoán (Predictive Analytics - dự báo tương lai dựa trên quá khứ) sang Phân tích theo quy định (Prescriptive Analytics - đưa ra hành động, chính sách cụ thể dựa trên dự đoán).

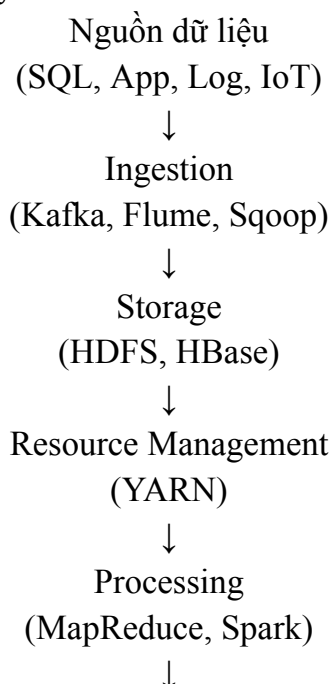
Đường ống dữ liệu cho Big Data

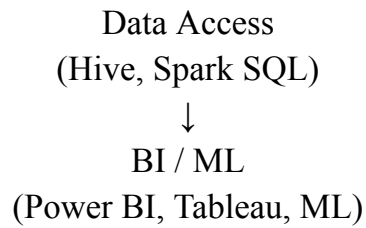


→ Kiến trúc tổng quan của một hệ thống Big Data. Dữ liệu được

- + Thu thập từ nhiều nguồn khác nhau thông qua các công cụ ingestion (Kafka, Flume, Sqoop)
- + Lưu trữ trên các hệ thống phân tán (HDFS, NoSQL), xử lý bằng Spark hoặc MapReduce
- + Sau đó được khai thác thông qua các công cụ phân tích, Machine Learning và trực quan hóa dữ liệu như Tableau hoặc Kibana.

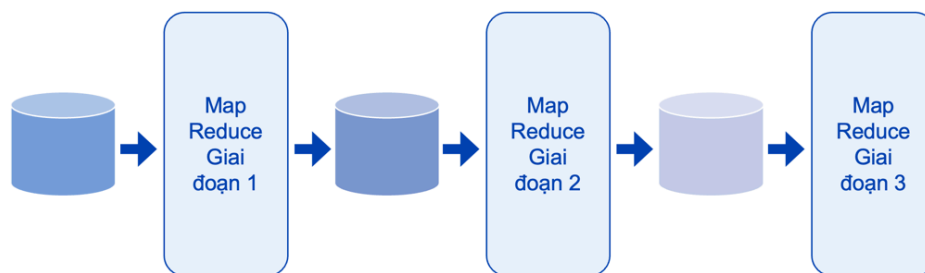
→ Phản ánh đầy đủ đường ống xử lý dữ liệu lớn (Big Data Pipeline) và vai trò của các thành phần trong Hadoop Ecosystem.





Vấn đề về MapReduce

- Một công việc MapReduce thường bao gồm nhiều chu trình Map và Reduce.
- Mỗi chu kỳ Map yêu cầu đọc từ HDFS và ghi vào đĩa cục bộ.
- Mỗi chu kỳ Reduce yêu cầu đọc từ đĩa cục bộ và ghi vào HDFS.
- Đĩa I/O rất ĐẮT.



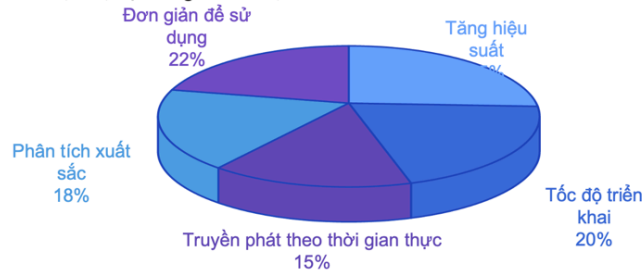
Apache Spark ra đời khi chi phí RAM giảm mạnh và các hệ điều hành 64-bit cho phép sử dụng lượng bộ nhớ rất lớn. Thay vì liên tục đọc ghi dữ liệu xuống đĩa như Hadoop MapReduce, Spark xử lý và lưu dữ liệu trung gian trực tiếp trong bộ nhớ (In-Memory Computing), giúp giảm Disk I/O và đạt hiệu năng cao hơn nhiều lần so với MapReduce truyền thống.

Spark

Apache Spark là framework xử lý dữ liệu phân tán được xây dựng dựa trên ý tưởng xử lý song song tương tự MapReduce nhưng tối ưu hơn về hiệu năng.

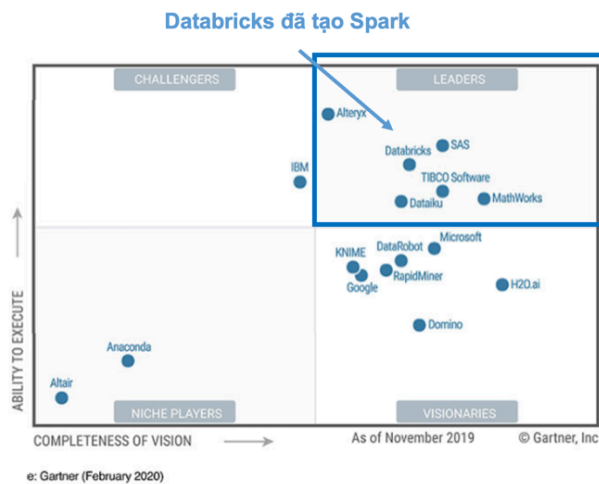
Thực hiện tính toán dựa trên RDD (Resilient Distributed Dataset) và lưu dữ liệu trung gian trong bộ nhớ (In-Memory) thay vì liên tục ghi xuống đĩa như Hadoop MapReduce.

- Apache Spark là công cụ xử lý thể hệ tiếp theo trong Hệ sinh thái Hadoop
- Nó được sử dụng rộng rãi cho nhiều nhiệm vụ khác nhau:
 - ETL và làm sạch dữ liệu
 - Công việc hàng loạt SQL trên tập dữ liệu lớn
 - Xử lý dữ liệu phát trực tuyến từ cảm biến, IoT, hệ thống tài chính, v.v.
 - Học máy và suy luận



Gartner Magic Quadrant

- Gartner Magic Quadrant là một nguồn phổ biến để xem xét các xu hướng công nghệ
- Họ xem xét các sản phẩm dựa trên tầm nhìn đầy đủ và khả năng thực hiện
- Các sản phẩm ở góc phần tư trên cùng bên phải là những sản phẩm hoàn chỉnh và được thực hiện tốt nhất
- Apache Spark được đánh giá cao về khả năng hợp nhất dữ liệu và công việc máy học tải tất cả trong một nền tảng duy nhất
- Tích hợp từ đầu đến cuối của khám phá dữ liệu, tiền xử lý dữ liệu, học máy để suy luận học máy



Kiến trúc Apache Spark

Apache Spark tuân theo mô hình Master-Slave. Các thành phần chính:

- Driver Program: Tiến trình trung tâm điều phối toàn bộ ứng dụng Spark.
 - Tạo SparkContext hoặc SparkSession
 - Lập kế hoạch thực thi
 - Chia dữ liệu thành các partition
 - Gửi task cho Executor
 - Thu thập kết quả cuối cùng
- Executor: Các tiến trình chạy trên Worker Node.
 - Thực hiện các phép tính
 - Lưu dữ liệu trung gian trong bộ nhớ
 - Trả kết quả về Driver

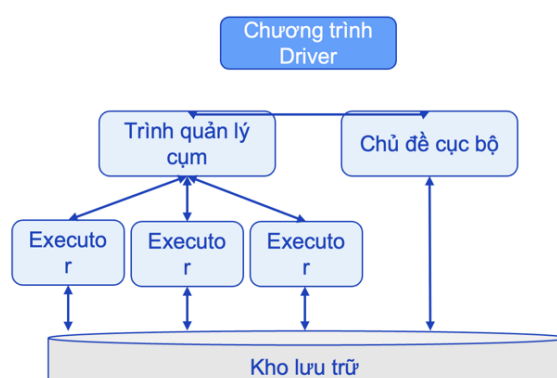
- Cluster Manager: Quản lý tài nguyên của cluster (YARN, Standalone Cluster Manager,..)

Driver	Executor
Điều phối công việc	Thực hiện công việc
Tạo DAG	Chạy Task
Thu thập kết quả	Xử lý dữ liệu
1 Driver/App	Nhiều Executor

Spark Driver và các Executor (1/2)

- Apache Spark tuân theo kiểu kiến trúc master-slave
- Chương trình Driver đóng vai trò là chủ và điều phối việc tạo hoặc loại bỏ các Executor
- Trình điều khiển phân vùng dữ liệu và chuyển từng phân vùng cho một Executor để xử lý phân tán song song
- Mỗi Executor thực hiện tính toán thực tế trên phân vùng dữ liệu của nó
- Executor truyền lại kết quả cho

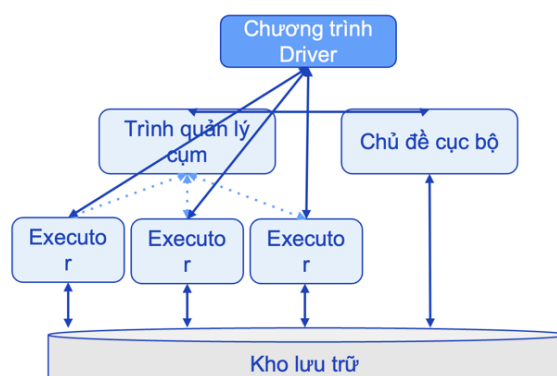
Driver
Samsung Innovation Campus



Chương 6. Xử lý Big Data với Apache Spark 13

- Apache Spark tuân theo kiểu kiến trúc master-slave
- Chương trình Driver đóng vai trò là chủ và điều phối việc tạo hoặc loại bỏ các Executor
- Trình điều khiển phân vùng dữ liệu và chuyển từng phân vùng cho một Executor để xử lý phân tán song song
- Mỗi Executor thực hiện tính toán thực tế trên phân vùng dữ liệu của nó
- Executor truyền lại kết quả cho

Driver
Samsung Innovation Campus



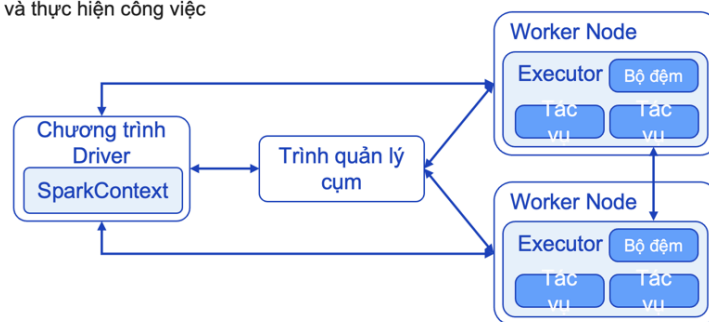
Chương 6. Xử lý Big Data với Apache Spark 14

SparkContext chính là điểm vào chính (main entry point) cho mọi hàm và tác vụ của Spark. Nó đại diện cho kết nối vật lý giữa chương trình với toàn bộ cụm Spark (Spark Cluster).

→ Cơ chế hoạt động: Chương trình Driver sử dụng SparkContext để thiết lập giao tiếp trực tiếp với Trình quản lý cụm nhằm điều phối tài nguyên, đồng thời phân bổ các Tác vụ (Tasks) xuống các Bộ đệm (Cache) bên trong từng Worker Node.

Đối tượng điểm vào SparkContext

- Đối tượng SparkContext là điểm vào chính cho hàm Spark.
- Nó đại diện cho kết nối với cụm Spark
- Chương trình Trình điều khiển sử dụng SparkContext để thiết lập giao tiếp với người quản lý tài nguyên và cụm nhằm điều phối và thực hiện công việc



Các đối tượng điểm vào khác

- Đối với các phiên bản Spark trước Spark 2.x, có một số đối tượng điểm nhập bổ sung
- SQLContext
 - Điểm vào Spark SQL và cung cấp chức năng SQL cơ bản
 - Thực hiện các hoạt động giống như SQL trên Bộ dữ liệu và Khung dữ liệu
- HiveContext
 - Ngoài chức năng SQL cơ bản, cho phép giao tiếp với Apache Hive
 - Sử dụng thay cho SQLContext khi chức năng Hive là cần thiết
- Spark Streaming Context
 - Điểm vào chính cho tất cả các chức năng truyền phát
 - Sử dụng bởi Spark Dstream API
- Post Spark 2.x, SparkSession cung cấp một đối tượng điểm vào thống nhất
 - Sử dụng SparkSession để tạo SparkContext

→ Từ phiên bản Spark 2.x trở đi: Xuất hiện một đối tượng hợp nhất tên là SparkSession. Nó đóng vai trò là "điểm vào thống nhất", giúp khởi tạo SparkContext bên dưới hoặc trực tiếp tích hợp luôn SQL/Hive mà không cần phải khai báo nhiều đối tượng phức tạp như trước nữa.

Bộ nhớ cục bộ	Bộ nhớ phân tán
---------------	-----------------

Spark Submit

Là công cụ dùng để triển khai và chạy ứng dụng Spark trên cluster. Cú pháp cơ bản:

```
spark-submit \
```

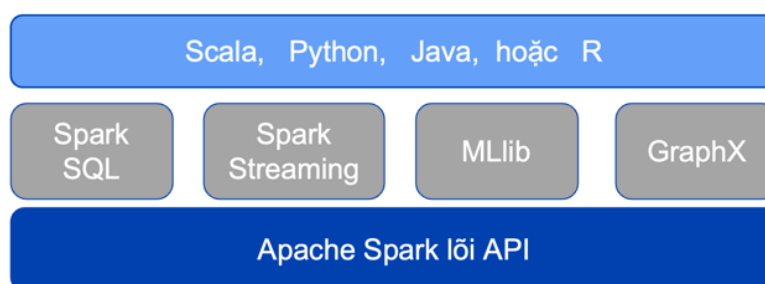
```
--master yarn \
```

```
--deploy-mode cluster \
```

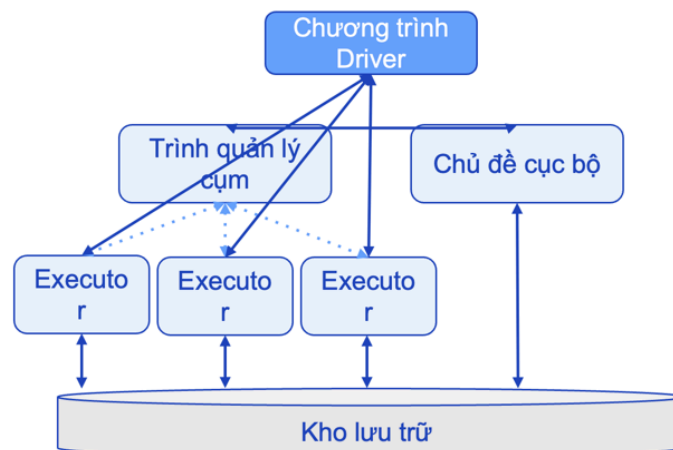
```
my_app.py
```

Apache Spark cung cấp nhiều API lập trình giải quyết các lĩnh vực khác nhau

- Lỗi API: Tốt cho xử lý dữ liệu phi cấu trúc và bán cấu trúc
- Spark SQL: Khung dữ liệu API, tốt cho việc xử lý dữ liệu có cấu trúc
- Spark Streaming - API truyền trực tuyến có cấu trúc và Dstream (một chuỗi liên tục các RDD thể hiện luồng dữ liệu theo thời gian thực), tốt cho việc xử lý dữ liệu thời gian thực
- ML và Mllib API: Học máy Spark
- GraphX API: Xử lý các nút và cạnh



- Apache Spark tuân theo kiểu kiến trúc master-slave.
- Chương trình Driver đóng vai trò là chủ và điều phối việc tạo hoặc loại bỏ các Executor.
- Trình điều khiển phân vùng dữ liệu và chuyển từng phân vùng cho một Executor để xử lý phân tán song song.
- Mỗi Executor thực hiện tính toán thực tế trên phân vùng dữ liệu của nó. Executor truyền lại kết quả cho Driver



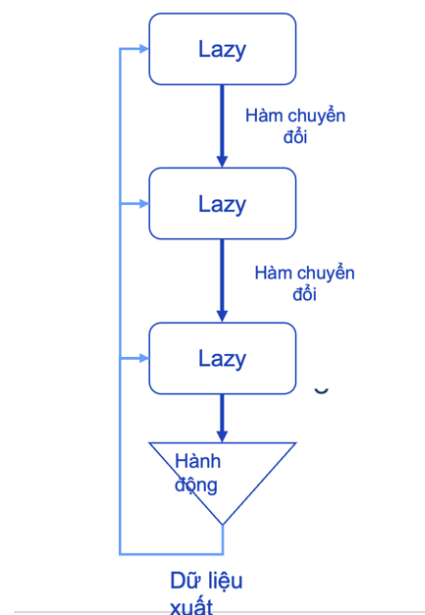
Apache Spark cung cấp môi trường shell để phát triển tương tác dễ dàng. Shell cung cấp môi trường REPL (Đọc đánh giá vòng lặp in) nơi các nhà phát triển có thể nhận được kết quả trả về ngay lập tức mà không cần phải biên dịch mã của họ.

Tự động tạo các đối tượng mục nhập cốt lõi như Spark Context và Spark Session.

Chỉ khả dụng cho Scala và Python - không khả dụng cho Java và R.

Lazy Execution

Tất cả các phép Transformation trong Spark đều được thực hiện theo cơ chế Lazy Execution. Cho đến khi một hành động được gọi, yêu cầu một giá trị, tất cả các phép biến đổi đều bị tạm dừng.



Đồ thị theo chu kỳ có hướng (DAG) trong đó mỗi nút biểu thị một RDD và mỗi cạnh biểu thị một thao tác chuyển đổi

- Khi Spark đạt đến RDD không được đánh giá cao nhất, nó bắt đầu thực hiện các phép biến đổi thực tế.
- Mỗi chuyển đổi sẽ kích hoạt RDD tiếp theo được tạo cho đến khi đầu ra hành động cuối cùng được tạo ra.

Transformation và Action

- Transformation tạo ra RDD mới từ RDD hiện có.
- Action kích hoạt quá trình thực thi và trả về kết quả.

Chuyển đổi dữ liệu với Core API

Hàm ẩn danh trong Python cho phép chúng ta xác định hàm một cách nhanh chóng mà không cần xác định nó, sử dụng ký hiệu lambda.

```
def my_func(str):
    return str.lower()
a_function(my_func)          OR          a_function(lambda str:
str.lower())
```

VD: def square(x):

return x*x

→ lambda x: x*x

Chuyển đổi RDD cơ bản

- Chuyển đổi của một tập dữ liệu

Chuyển đổi	Ý nghĩa
map(func)	Trả về tập dữ liệu phân tán mới được hình thành bằng cách chuyển từng phần tử của nguồn thông qua hàm func
filter(func)	Trả về tập dữ liệu mới được hình thành bằng cách chọn các phần tử đó của nguồn mà func trả về true.
distinct()	Trả về tập dữ liệu mới chứa các thành phần riêng biệt của tập dữ liệu nguồn
flatMap(func)	Tương tự như bản đồ, nhưng mỗi mục đầu vào có thể được ánh xạ tới 0 hoặc nhiều mục đầu ra (vì vậy func phải trả về một Seq thay vì một mục duy nhất)

Hành động RDD

- Các hành động không yêu cầu hàm làm tham số

Hành động	Ý nghĩa
collect()	Trả về tất cả các phần tử của tập dữ liệu dưới dạng một mảng tại chương trình trình điều khiển. Điều này thường hữu ích sau khi bộ lọc hoặc hoạt động khác trả về một tập hợp con đủ nhỏ của dữ liệu
count()	Trả về số lượng phần tử trong tập dữ liệu
first()	Trả về phần tử đầu tiên của tập dữ liệu (tương tự như take(1))
take(n)	Trả về một mảng có n phần tử đầu tiên của tập dữ liệu
saveAsTextFile(path)	Viết các thành phần của tập dữ liệu dưới dạng tệp văn bản (hoặc tập hợp các tệp văn bản) trong một thư mục nhất định trong hệ thống tệp cục bộ, HDFS hoặc bất kỳ hệ thống tệp nào khác được Hadoop hỗ trợ. Spark sẽ gọi toString trên từng phần tử để chuyển đổi nó thành một dòng văn bản trong tệp

Hành động	Ý nghĩa
reduce(func)	Tổng hợp các thành phần của tập dữ liệu bằng hàm func (nhận hai đối số và trả về một đối số). Hàm phải có tính chất giao hoán và kết hợp để có thể tính toán song song một cách chính xác
foreach(func)	Chạy một hàm func trên từng phần tử của tập dữ liệu. Điều này thường được thực hiện đối với các tác dụng phụ như cập nhật Bộ tích lũy hoặc tương tác với hệ thống lưu trữ bên ngoài
foreachPartition(func)	Chạy một hàm func trên mỗi phân vùng của tập dữ liệu
getNumPartitions()	Trả về số phân vùng

- Các phép biến đổi hoạt động ở cấp độ phân vùng, thay vì cấp độ hàng

Chuyển đổi	Ý nghĩa
mapPartitions(func)	Tương tự như bản đồ, nhưng chạy riêng trên từng phân vùng (khối) của RDD, vì vậy func phải thuộc loại <code>Iterator<T> => Iterator<U></code> khi chạy trên RDD thuộc loại T
mapPartitionsWithIndex(func)	Tương tự như mapPartitions, nhưng cũng cung cấp cho func một giá trị số nguyên biểu thị chỉ số của phân vùng, vì vậy func phải thuộc loại <code>(Int, Iterator<T>) => Iterator<U></code> khi chạy trên RDD thuộc loại T

Chuyển đổi	Ý nghĩa
coalesce(numPartitions)	Giảm số lượng phân vùng trong RDD xuống numPartitions. Hữu ích để chạy các hoạt động hiệu quả hơn sau khi lọc một tập dữ liệu lớn.
repartition(numPartitions)	Xáo trộn lại dữ liệu trong RDD một cách ngẫu nhiên để tạo nhiều hoặc ít phân vùng hơn và cân bằng dữ liệu trên chúng. Điều này luôn xào trộn tất cả dữ liệu qua mạng.
repartitionAndSortWithinPartitions(partitioner)	Phân vùng lại RDD theo trình phân vùng đã cho và trong mỗi phân vùng kết quả, hãy sắp xếp các bản ghi theo khóa của chúng. Điều này hiệu quả hơn so với việc gọi phân vùng lại và sau đó sắp xếp trong từng phân vùng vì nó có thể đẩy quá trình sắp xếp xuống máy xào trộn.

Pair RDD

Pair RDD là RDD có dạng Key-Value.

→ Tạo Pair RDD

- Cũng như các hoạt động RDD khác, bước đầu tiên bắt buộc là tạo Pair RDD
- Các phép biến đổi hữu ích: **map()**, **flatMap()**, **keyBy()**, **flatMapValues()**

Chuyển đổi FlatMap

- Trả về một tập hợp hoặc chuỗi
 - Lấy từng mục trong bộ sưu tập hoặc chuỗi và xuất ra dưới dạng một phần tử riêng biệt
- + **map** trả về bộ sưu tập đó
- + **FlatMap** lấy từng mục trong bộ sưu tập và xuất nó dưới dạng một phần tử riêng biệt

<pre>my_fruits = ["apples oranges pear banana"] dataRDD = sc.parallelize(my_fruits) print("Number of elements is this RDD:", dataRDD.count()) print("It contains the following string:", dataRDD.collect())</pre>	Input
<pre>Number of elements is this RDD: 1 It contains the following string: ['apples oranges pear banana']</pre>	Output
<pre>mapRDD = dataRDD.map(lambda line: line.split(' ')) for line in mapRDD.collect(): print(line)</pre>	Input
<pre>['apples', 'oranges', 'pear', 'banana']</pre>	Output
<pre>flatMapRDD = dataRDD.flatMap(lambda line: line.split(' ')) for line in flatMapRDD.collect(): print(line)</pre>	Input
<pre>apples oranges pear banana</pre>	Output

Chuyển đổi	Ý nghĩa
<code>reduceByKey(func)</code>	Khi được gọi trên tập dữ liệu gồm các cặp (K, V), trả về tập dữ liệu gồm các cặp (K, V) trong đó các giá trị cho mỗi khóa được tổng hợp bằng hàm giảm đã cho, hàm này phải thuộc loại (V,V) => V.
<code>aggregateByKey(zeroValue) (seqOp, combOp)</code>	Khi được gọi trên tập dữ liệu gồm các cặp (K, V), trả về tập dữ liệu gồm các cặp (K, U) trong đó các giá trị cho mỗi khóa được tổng hợp bằng cách sử dụng các hàm kết hợp đã cho và giá trị "không" trung lập. Cho phép loại giá trị tổng hợp khác với loại giá trị đầu vào, đồng thời tránh phân bổ không cần thiết.
<code>groupByKey()</code>	Khi được gọi trên tập dữ liệu gồm (K, V) cặp, trả về tập dữ liệu gồm (K, Iterable<V>) cặp. Nhóm hiệu quả tất cả các giá trị của cùng một khóa
<code>sortByKey([ascending])</code>	Khi được gọi trên tập dữ liệu gồm (K, V) cặp trong đó K triển khai Ordered, trả về tập dữ liệu gồm (K, V) cặp được sắp xếp theo khóa theo thứ tự tăng dần hoặc giảm dần, như được chỉ định trong đối số boolean tăng dần.
<code>join(otherDataset)</code>	Khi được gọi trên các tập dữ liệu loại (K, V) và (K, W), trả về tập dữ liệu gồm các cặp (K, (V, W)) với tất cả các cặp phần tử cho mỗi khóa. Các liên kết bên ngoài được hỗ trợ thông qua <code>leftOuterJoin</code> , <code>rightOuterJoin</code> và <code>fullOuterJoin</code> .

Chuyển đổi RDD hỗn hợp

Chuyển đổi	Ý nghĩa
<code>sample(withReplacement, fraction, seed)</code>	Lấy mẫu phân số có Thay thế dữ liệu, có hoặc không thay thế, bằng cách sử dụng hạt giống trình tạo số ngẫu nhiên nhất định.
<code>pipe(command, [envVars])</code>	Đưa từng phân vùng của RDD thông qua lệnh shell, ví dụ: một tập lệnh Perl hoặc bash. Các phần tử RDD được ghi vào stdin của quy trình và các dòng xuất ra thiết bị xuất chuẩn của nó được trả về dưới dạng RDD của các chuỗi.

Broadcast Variable

Cho phép chia sẻ dữ liệu chỉ đọc tới tất cả Executor, giúp tránh việc gửi lặp lại cùng dữ liệu cho nhiều task.

```
countries = sc.broadcast(country_dict)
```

Accumulator

Là biến dùng để tổng hợp dữ liệu từ nhiều Executor.

```
counter = sc.accumulator(0)
```

→ Các Executor có thể tăng giá trị của Accumulator.
Driver có thể đọc kết quả cuối cùng.

HiveQL: Ngôn ngữ truy vấn của Hive, cú pháp tương tự SQL. HiveQL được chuyển đổi thành các job MapReduce hoặc Spark để thực thi trên cluster.

Hive Metastore: Nơi lưu trữ metadata của Hive. Bao gồm: Database, Table, Column, Partition, Location.

→ *Hive không lưu dữ liệu thực tế. Hive chỉ quản lý Schema và Metadata. Dữ liệu thực tế được lưu trong HDFS.*

Beeline Client

Là công cụ dòng lệnh dùng để kết nối và truy vấn HiveServer2.

```
beeline -u jdbc:hive2://localhost:10000
```

JDBC Connection

Các ứng dụng Java có thể truy cập Hive thông qua JDBC.

```
jdbc:hive2://server:10000/default
```

JDBC giúp các ứng dụng bên ngoài tích hợp với Hive giống như các hệ quản trị cơ sở dữ liệu quan hệ.

Spark SQL

Spark SQL là module của Apache Spark dùng để xử lý dữ liệu có cấu trúc. Spark SQL sử dụng DataFrame, Dataset và Catalyst Optimizer để tối ưu hiệu năng truy vấn.

- DataFrame: tập dữ liệu dạng bảng gồm rows, columns, schema

- Đọc CSV

```
df = spark.read.csv(
    "sales.csv",
    header=True,
    inferSchema=True
)
```

- Đọc Hive Table

```
df = spark.table("sales")
```

Chuyển DataFrame thành RDD

```
rdd = df.rdd
```

Chuyển RDD thành DataFrame

```
df = spark.createDataFrame(rdd)
```

- Ghi dữ liệu xuống Hive: `df.write.saveAsTable("sales")`

Spark Debugging

- Log Driver chứa lỗi ứng dụng, DAG và tiến trình thực thi. Đây là nơi nên kiểm tra đầu tiên khi debug.
- Log Executor chứa lỗi task, lỗi partition và lỗi xử lý dữ liệu.

Thông thường không nên ghi log quá nhiều trong Executor vì có thể sinh ra lượng log rất lớn trên cluster.

Spark Persistence

- Cache - `df.cache()`: Lưu dữ liệu trong bộ nhớ để tái sử dụng.
- Persist - `df.persist()`: Cho phép lựa chọn nhiều cơ chế lưu trữ: Memory, Disk

Virtual Box

Một ứng dụng ảo hóa đa nền tảng chạy trên Windows, Linux, Mac OS X, cho phép nhiều máy cùng chạy trên một hệ thống

